

EXPERIMENT 8

IMPLEMENTATION OF DES ALGORITHM

AIM

To implement the Data Encryption Standard (DES) algorithm in C and perform encryption and decryption of a 64-bit plaintext using a 64-bit key.

Algorithm

1. Read the plaintext and encryption key, each as a 16-digit hexadecimal value.
2. Generate 16 round keys from the given 64-bit key.
3. Apply the Initial Permutation (IP) to the plaintext.
4. Divide the permuted data into two 32-bit halves: L and R.
5. Perform 16 Feistel rounds:
 - Expand the 32-bit right half to 48 bits.
 - XOR it with the round key.
 - Apply S-box substitution.
 - Apply the P-box permutation.
 - XOR the result with the left half.
 - Swap the two halves.
6. Apply the Final Permutation (FP) after the 16 rounds.
7. Display the encrypted ciphertext.
8. For decryption, use the same process with the round keys in reverse order.
9. Display the recovered plaintext.

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
#include <string.h>
```

```
static const int IP[64] = {  
    58,50,42,34,26,18,10,2,60,52,44,36,28,20,12,4,  
    62,54,46,38,30,22,14,6,64,56,48,40,32,24,16,8,  
    57,49,41,33,25,17,9,1,59,51,43,35,27,19,11,3,  
    61,53,45,37,29,21,13,5,63,55,47,39,31,23,15,7  
};
```

```
static const int FP[64] = {  
    40,8,48,16,56,24,64,32,39,7,47,15,55,23,63,31,  
    38,6,46,14,54,22,62,30,37,5,45,13,53,21,61,29,  
    36,4,44,12,52,20,60,28,35,3,43,11,51,19,59,27,  
    34,2,42,10,50,18,58,26,33,1,41,9,49,17,57,25  
};
```

```
static const int PC1[56] = {  
    57,49,41,33,25,17,9,1,58,50,42,34,26,18,10,2,  
    59,51,43,35,27,19,11,3,60,52,44,36,28,20,12,4,  
    61,53,45,37,29,21,13,5,62,54,46,38,30,22,14,6,  
    63,55,47,39,31,23,15,7  
};
```

```
static const int PC2[48] = {  
    14,17,11,24,1,5,3,28,15,6,21,10,23,19,12,4,  
    26,8,16,7,27,20,13,2,41,52,31,37,47,55,30,40,  
    51,45,33,48,44,49,39,56,34,53,46,42,50,36,29,32  
};
```

```
static const int E[48] = {  
    32,1,2,3,4,5,4,5,6,7,8,9,8,9,10,11,12,13,  
    12,13,14,15,16,17,16,17,18,19,20,21,20,21,22,23,  
    24,25,24,25,26,27,28,29,28,29,30,31,32,1  
};
```

```
static const int P[32] = {  
    16,7,20,21,29,12,28,17,1,15,23,26,5,18,31,10,
```

```
2,8,24,14,32,27,3,9,19,13,30,6,22,11,4,25  
};
```

```
static const int SHIFTS[16] = {  
    1,1,2,2,2,2,2,2,1,2,2,2,2,2,1  
};
```

```
static const int S[8][4][16] = {  
    {  
        {14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7},  
        {0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8},  
        {4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0},  
        {15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13}  
    },  
    {  
        {15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10},  
        {3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5},  
        {0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15},  
        {13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9}  
    },  
    {  
        {10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8},  
        {13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1},  
        {13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7},  
        {1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12}  
    },  
    {  
        {7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15},  
        {13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9},
```

```

    {10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4},
    {3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14}
},
{
    {2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9},
    {14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6},
    {4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14},
    {11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3}
},
{
    {12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11},
    {10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8},
    {9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6},
    {4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13}
},
{
    {4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1},
    {13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6},
    {1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2},
    {6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12}
},
{
    {13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7},
    {1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2},
    {7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8},
    {2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11}
}
};

```

```

uint64_t permute(uint64_t input, const int *table, int n, int inputBits) {
    uint64_t output = 0;
    int i;

    for (i = 0; i < n; i++) {
        output <<= 1;
        output |= (input >> (inputBits - table[i])) & 1ULL;
    }

    return output;
}

```

```

uint32_t leftRotate28(uint32_t x, int shift) {
    return ((x << shift) | (x >> (28 - shift))) & 0xFFFFFFFF;
}

```

```

void generateKeys(uint64_t key, uint64_t roundKeys[16]) {
    uint64_t permutedKey;
    uint32_t C, D;
    int i;

    permutedKey = permute(key, PC1, 56, 64);

    C = (uint32_t)((permutedKey >> 28) & 0xFFFFFFFF);
    D = (uint32_t)(permutedKey & 0xFFFFFFFF);

    for (i = 0; i < 16; i++) {
        C = leftRotate28(C, SHIFTS[i]);
        D = leftRotate28(D, SHIFTS[i]);
    }
}

```

```

    uint64_t combined = ((uint64_t)C << 28) | D;
    roundKeys[i] = permute(combined, PC2, 48, 56);
}
}

uint32_t feistel(uint32_t R, uint64_t roundKey) {
    uint64_t expanded;
    uint64_t xored;
    uint32_t sbboxOutput = 0;
    uint32_t result;
    int i;

    expanded = permute((uint64_t)R, E, 48, 32);
    xored = expanded ^ roundKey;

    for (i = 0; i < 8; i++) {
        int shift = 42 - (i * 6);
        int sixBits = (int)((xored >> shift) & 0x3F);

        int row = ((sixBits & 0x20) >> 4) | (sixBits & 0x01);
        int col = (sixBits >> 1) & 0x0F;

        sbboxOutput = (sbboxOutput << 4) | S[i][row][col];
    }

    result = (uint32_t)permute(sbboxOutput, P, 32, 32);

    return result;
}

```

```
}
```

```
uint64_t DES(uint64_t block, uint64_t roundKeys[16], int decrypt) {
```

```
    uint64_t ip;
```

```
    uint32_t L, R;
```

```
    int i;
```

```
    ip = permute(block, IP, 64, 64);
```

```
    L = (uint32_t)(ip >> 32);
```

```
    R = (uint32_t)(ip & 0xFFFFFFFF);
```

```
    for (i = 0; i < 16; i++) {
```

```
        uint32_t oldR = R;
```

```
        int keyIndex = decrypt ? (15 - i) : i;
```

```
        R = L ^ feistel(R, roundKeys[keyIndex]);
```

```
        L = oldR;
```

```
    }
```

```
    uint64_t combined = ((uint64_t)R << 32) | L;
```

```
    return permute(combined, FP, 64, 64);
```

```
}
```

```
int main() {
```

```
    uint64_t plaintext, key;
```

```
    uint64_t roundKeys[16];
```

```
    uint64_t ciphertext, decrypted;
```

```

printf("DES Encryption and Decryption\n");
printf("-----\n");

printf("Enter 16-digit hexadecimal plaintext: ");
if (scanf("%16llx", (unsigned long long *)&plaintext) != 1) {
    printf("Invalid plaintext.\n");
    return 1;
}

printf("Enter 16-digit hexadecimal key: ");
if (scanf("%16llx", (unsigned long long *)&key) != 1) {
    printf("Invalid key.\n");
    return 1;
}

generateKeys(key, roundKeys);

ciphertext = DES(plaintext, roundKeys, 0);
decrypted = DES(ciphertext, roundKeys, 1);

printf("\nPlaintext : %016llX",
    (unsigned long long)plaintext);

printf("\nKey      : %016llX",
    (unsigned long long)key);

printf("\nCiphertext : %016llX",
    (unsigned long long)ciphertext);

```



```

printf("\nDecrypted : %016llx\n",
      (unsigned long long)decrypted);

return 0;
}

```

OUTPUT:

The screenshot shows a C code editor with a file named `main.c`. The code implements a DES encryption and decryption process. The output window on the right shows the execution results, including the input plaintext and key, the resulting ciphertext, and the decrypted plaintext, which matches the original input.

```

main.c
198 return 1;
199 }
200
201 generateKeys(key, roundKeys);
202
203 ciphertext = DES(plaintext, roundKeys, 0);
204 decrypted = DES(ciphertext, roundKeys, 1);
205
206 printf("\nPlaintext : %016llx",
207        (unsigned long long)plaintext);
208
209 printf("\nKey : %016llx",
210        (unsigned long long)key);
211
212 printf("\nCiphertext : %016llx",
213        (unsigned long long)ciphertext);
214
215 printf("\nDecrypted : %016llx\n",
216        (unsigned long long)decrypted);
217
218 return 0;

```

Output

```

DES Encryption and Decryption
-----
Enter 16-digit hexadecimal plaintext: 123456ABCD132536
Enter 16-digit hexadecimal key: AAB09182736CCDD

Plaintext : 123456ABCD132536
Key : AAB09182736CCDD
Ciphertext : 7ABB66DB1438E354
Decrypted : 123456ABCD132536

=== Code Execution Successful ===

```

RESULT:

Thus, the DES algorithm was successfully implemented in C, and the given plaintext was encrypted into ciphertext and successfully decrypted back to the original plaintext.

EXPERIMENT 9

IMPLEMENTATION OF 2DES (DOUBLE DES)

AIM:

To implement the Double Data Encryption Standard (2DES) algorithm in C by performing DES encryption twice using two different keys, and to decrypt the ciphertext back to the original plaintext.

ALGORITHM:

1. Read the plaintext and two 64-bit encryption keys.
2. Generate the DES round keys for Key 1 and Key 2.
3. Encrypt the plaintext using Key 1 to obtain the intermediate ciphertext.
4. Encrypt the intermediate ciphertext using Key 2 to obtain the final ciphertext.
5. For decryption:
 - Decrypt the final ciphertext using Key 2.
 - Decrypt the resulting data using Key 1.
6. Display the plaintext, intermediate ciphertext, final ciphertext, and decrypted plaintext.

ENCRYPTION:

Plaintext
↓
DES Encryption using K1
↓
Intermediate Ciphertext
↓
DES Encryption using K2
↓
Final Ciphertext

DECRYPTION:

Final Ciphertext
↓
DES Decryption using K2
↓
Intermediate Ciphertext
↓
DES Decryption using K1
↓
Original Plaintext

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
/* Initial Permutation */
```

```
int IP[64] = {  
    58,50,42,34,26,18,10,2,  
    60,52,44,36,28,20,12,4,  
    62,54,46,38,30,22,14,6,  
    64,56,48,40,32,24,16,8,  
    57,49,41,33,25,17,9,1,  
    59,51,43,35,27,19,11,3,  
    61,53,45,37,29,21,13,5,  
    63,55,47,39,31,23,15,7  
};
```

```
/* Final Permutation */
```

```
int FP[64] = {  
    40,8,48,16,56,24,64,32,  
    39,7,47,15,55,23,63,31,  
    38,6,46,14,54,22,62,30,  
    37,5,45,13,53,21,61,29,  
    36,4,44,12,52,20,60,28,  
    35,3,43,11,51,19,59,27,  
    34,2,42,10,50,18,58,26,  
    33,1,41,9,49,17,57,25  
};
```

```
/* PC-1 */
```

```
int PC1[56] = {  
    57,49,41,33,25,17,9,  
    1,58,50,42,34,26,18,  
    10,2,59,51,43,35,27,  
    19,11,3,60,52,44,36,  
    63,55,47,39,31,23,15,  
    7,62,54,46,38,30,22,  
    14,6,61,53,45,37,29,  
    21,13,5,28,20,12,4  
};
```

```
/* PC-2 */
```

```
int PC2[48] = {  
    14,17,11,24,1,5,  
    3,28,15,6,21,10,  
    23,19,12,4,26,8,  
    16,7,27,20,13,2,  
    41,52,31,37,47,55,  
    30,40,51,45,33,48,  
    44,49,39,56,34,53,  
    46,42,50,36,29,32  
};
```

```
/* Expansion */
```

```
int E[48] = {  
    32,1,2,3,4,5,  
    4,5,6,7,8,9,  
    8,9,10,11,12,13,  
    12,13,14,15,16,17,
```

```
    16,17,18,19,20,21,  
    20,21,22,23,24,25,  
    24,25,26,27,28,29,  
    28,29,30,31,32,1  
};
```

```
/* P Permutation */  
int P[32] = {  
    16,7,20,21,29,12,28,17,  
    1,15,23,26,5,18,31,10,  
    2,8,24,14,32,27,3,9,  
    19,13,30,6,22,11,4,25  
};
```

```
/* Left shift schedule */  
int SHIFTS[16] = {  
    1,1,2,2,2,2,2,2,  
    1,2,2,2,2,2,2,1  
};
```

```
/* DES S-Boxes */  
int S[8][4][16] = {  
  
    {  
        {14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7},  
        {0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8},  
        {4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0},  
        {15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13}  
    },
```

{
{15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10},
{3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5},
{0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15},
{13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9}
},

{
{10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8},
{13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1},
{13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7},
{1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12}
},

{
{7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15},
{13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9},
{10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4},
{3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14}
},

{
{2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9},
{14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6},
{4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14},
{11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3}
},

```
{
{12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11},
{10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8},
{9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6},
{4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13}
},
```

```
{
{4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1},
{13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6},
{1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2},
{6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12}
},
```

```
{
{13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7},
{1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2},
{7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8},
{2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11}
}
};
```

```
/* Permutation function */
uint64_t permute(uint64_t input, int table[],
                 int n, int inputBits)
{
    uint64_t output = 0;
    int i;
```

```

for(i = 0; i < n; i++)
{
    output <<= 1;
    output |= (input >>
                (inputBits - table[i])) & 1ULL;
}

return output;
}

/* Left rotation */
uint32_t leftRotate28(uint32_t x, int shift)
{
    return ((x << shift) |
            (x >> (28 - shift))) & 0xFFFFFFFF;
}

/* Generate DES round keys */
void generateKeys(uint64_t key, uint64_t keys[16])
{
    uint64_t permutedKey;
    uint32_t C, D;
    int i;

    permutedKey =
        permute(key, PC1, 56, 64);

    C = (uint32_t)(permutedKey >> 28)
        & 0xFFFFFFFF;

```



```
D = (uint32_t)permutedKey  
    & 0xFFFFFFFF;
```

```
for(i = 0; i < 16; i++)  
{  
    C = leftRotate28(C, SHIFTS[i]);  
    D = leftRotate28(D, SHIFTS[i]);
```

```
uint64_t combined =  
    ((uint64_t)C << 28) | D;
```

```
keys[i] =  
    permute(combined, PC2, 48, 56);  
}  
}
```

```
/* Feistel function */
```

```
uint32_t feistel(uint32_t R,  
                 uint64_t roundKey)  
{  
    uint64_t expanded;  
    uint64_t xored;  
    uint32_t sboxOutput = 0;  
    int i;  
  
    expanded =  
        permute((uint64_t)R, E, 48, 32);
```

```

xored = expanded ^ roundKey;

for(i = 0; i < 8; i++)
{
    int shift = 42 - (i * 6);

    int sixBits =
        (int)((xored >> shift) & 0x3F);

    int row =
        ((sixBits & 0x20) >> 4)
        | (sixBits & 0x01);

    int col =
        (sixBits >> 1) & 0x0F;

    sbboxOutput =
        (sbboxOutput << 4)
        | S[i][row][col];
}

return (uint32_t)
    permute(sbboxOutput, P, 32, 32);
}

/* Single DES */
uint64_t DES(uint64_t block,
    uint64_t keys[16],
    int decrypt)

```

```

{
    uint64_t ip;
    uint32_t L, R;
    int i;

    ip = permute(block, IP, 64, 64);

    L = (uint32_t)(ip >> 32);
    R = (uint32_t)ip;

    for(i = 0; i < 16; i++)
    {
        uint32_t oldR = R;

        int keyIndex =
            decrypt ? 15 - i : i;

        R =
            L ^ feistel(R, keys[keyIndex]);

        L = oldR;
    }

    uint64_t combined =
        ((uint64_t)R << 32) | L;

    return permute(combined, FP, 64, 64);
}

```

```
/* Double DES */
uint64_t DoubleDES_Encrypt(
    uint64_t plaintext,
    uint64_t key1[],
    uint64_t key2[])
{
    uint64_t intermediate;

    intermediate =
        DES(plaintext, key1, 0);

    return DES(intermediate, key2, 0);
}
```

```
/* Double DES Decryption */
uint64_t DoubleDES_Decrypt(
    uint64_t ciphertext,
    uint64_t key1[],
    uint64_t key2[])
{
    uint64_t intermediate;

    intermediate =
        DES(ciphertext, key2, 1);

    return DES(intermediate, key1, 1);
}
```

```
int main()
```

```

{
    uint64_t plaintext;
    uint64_t key1, key2;
    uint64_t intermediate;
    uint64_t ciphertext;
    uint64_t decrypted;

    uint64_t keys1[16];
    uint64_t keys2[16];

    printf("DOUBLE DES ENCRYPTION AND DECRYPTION\n");
    printf("-----\n");

    printf("Enter 16-digit hexadecimal plaintext: ");
    scanf("%llx",
        (unsigned long long *)&plaintext);

    printf("Enter 16-digit hexadecimal Key 1: ");
    scanf("%llx",
        (unsigned long long *)&key1);

    printf("Enter 16-digit hexadecimal Key 2: ");
    scanf("%llx",
        (unsigned long long *)&key2);

    /* Generate keys */
    generateKeys(key1, keys1);
    generateKeys(key2, keys2);

```

```

/* First DES encryption */
intermediate =
    DES(plaintext, keys1, 0);

/* Second DES encryption */
ciphertext =
    DES(intermediate, keys2, 0);

/* Double DES decryption */
decrypted =
    DoubleDES_Decrypt(
        ciphertext, keys1, keys2);

printf("\nPlaintext      : %016llX",
        (unsigned long long)plaintext);

printf("\nKey 1          : %016llX",
        (unsigned long long)key1);

printf("\nKey 2          : %016llX",
        (unsigned long long)key2);

printf("\nAfter DES with K1  : %016llX",
        (unsigned long long)intermediate);

printf("\nFinal Ciphertext   : %016llX",
        (unsigned long long)ciphertext);

printf("\nDecrypted Plaintext : %016llX\n",

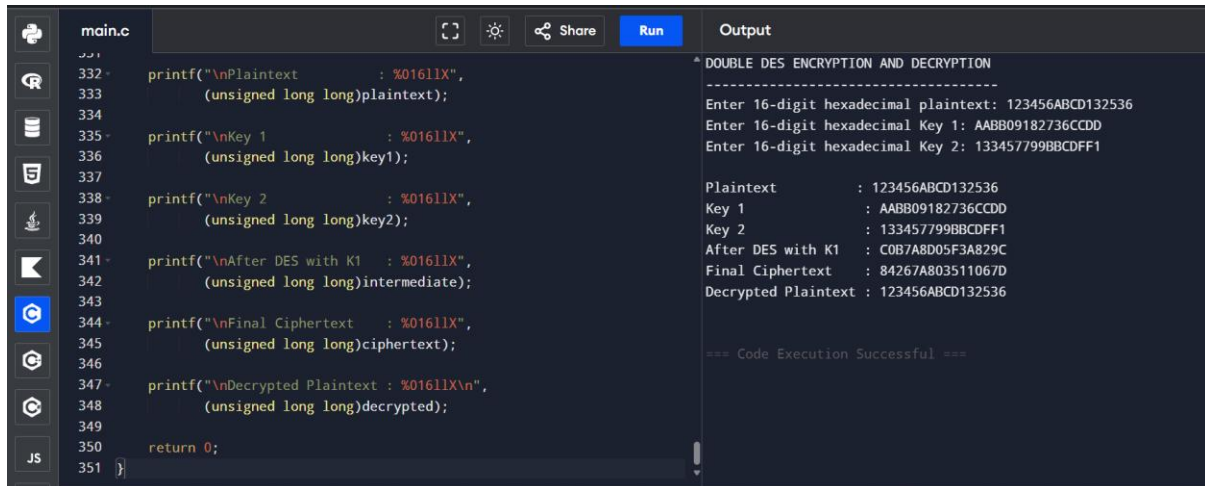
```

```
(unsigned long long)decrypted);
```

```
return 0;
```

```
}
```

OUTPUT:



```
main.c
332- printf("\nPlaintext      : %016lX",
333-        (unsigned long long)plaintext);
334-
335- printf("\nKey 1         : %016lX",
336-        (unsigned long long)key1);
337-
338- printf("\nKey 2         : %016lX",
339-        (unsigned long long)key2);
340-
341- printf("\nAfter DES with K1 : %016lX",
342-        (unsigned long long)intermediate);
343-
344- printf("\nFinal Ciphertext : %016lX",
345-        (unsigned long long)ciphertext);
346-
347- printf("\nDecrypted Plaintext : %016lX\n",
348-        (unsigned long long)decrypted);
349-
350- return 0;
351- }
```

Output

```
DOUBLE DES ENCRYPTION AND DECRYPTION
-----
Enter 16-digit hexadecimal plaintext: 123456ABCD132536
Enter 16-digit hexadecimal Key 1: AAB09182736CCDD
Enter 16-digit hexadecimal Key 2: 133457799BBCDFF1

Plaintext      : 123456ABCD132536
Key 1          : AAB09182736CCDD
Key 2          : 133457799BBCDFF1
After DES with K1 : COB7A8D05F3A829C
Final Ciphertext : 84267A803511067D
Decrypted Plaintext : 123456ABCD132536

=== Code Execution Successful ===
```

RESULT:

Thus, the Double DES (2DES) algorithm was successfully implemented in C by encrypting the plaintext twice using two different DES keys. The ciphertext was successfully decrypted to obtain the original plaintext.

EXPERIMENT 10

IMPLEMENTATION OF 3DES ALGORITHM

AIM:

To implement the Triple Data Encryption Standard (3DES) algorithm in C by performing DES encryption/decryption three times using three keys, and to recover the original plaintext through decryption.

ALGORITHM:

Encryption:

1. Read the 64-bit plaintext and three 64-bit keys K1, K2, and K3.
2. Generate the 16 DES round keys for each of the three keys.
3. Encrypt the plaintext using K1.
4. Decrypt the result using K2.
5. Encrypt the result again using K3.
6. The final result is the 3DES ciphertext.

The encryption process is:

Plaintext
↓
DES Encrypt using K1
↓
DES Decrypt using K2
↓
DES Encrypt using K3
↓
Ciphertext

Decryption:

1. Decrypt the ciphertext using K3.
2. Encrypt the result using K2.
3. Decrypt the result using K1.
4. The original plaintext is obtained.

Ciphertext
↓
DES Decrypt using K3
↓
DES Encrypt using K2

↓
DES Decrypt using K1
↓
Plaintext

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
/* Initial Permutation */
```

```
int IP[64] = {  
    58,50,42,34,26,18,10,2,  
    60,52,44,36,28,20,12,4,  
    62,54,46,38,30,22,14,6,  
    64,56,48,40,32,24,16,8,  
    57,49,41,33,25,17,9,1,  
    59,51,43,35,27,19,11,3,  
    61,53,45,37,29,21,13,5,  
    63,55,47,39,31,23,15,7  
};
```

```
/* Final Permutation */
```

```
int FP[64] = {  
    40,8,48,16,56,24,64,32,  
    39,7,47,15,55,23,63,31,  
    38,6,46,14,54,22,62,30,  
    37,5,45,13,53,21,61,29,  
    36,4,44,12,52,20,60,28,  
    35,3,43,11,51,19,59,27,  
    34,2,42,10,50,18,58,26,  
    33,1,41,9,49,17,57,25
```

```
};
```

```
/* Permuted Choice 1 */
```

```
int PC1[56] = {  
    57,49,41,33,25,17,9,  
    1,58,50,42,34,26,18,  
    10,2,59,51,43,35,27,  
    19,11,3,60,52,44,36,  
    63,55,47,39,31,23,15,  
    7,62,54,46,38,30,22,  
    14,6,61,53,45,37,29,  
    21,13,5,28,20,12,4  
};
```

```
/* Permuted Choice 2 */
```

```
int PC2[48] = {  
    14,17,11,24,1,5,  
    3,28,15,6,21,10,  
    23,19,12,4,26,8,  
    16,7,27,20,13,2,  
    41,52,31,37,47,55,  
    30,40,51,45,33,48,  
    44,49,39,56,34,53,  
    46,42,50,36,29,32  
};
```

```
/* Expansion */
```

```
int E[48] = {  
    32,1,2,3,4,5,
```

```
4,5,6,7,8,9,  
8,9,10,11,12,13,  
12,13,14,15,16,17,  
16,17,18,19,20,21,  
20,21,22,23,24,25,  
24,25,26,27,28,29,  
28,29,30,31,32,1  
};
```

```
/* P Permutation */  
int P[32] = {  
    16,7,20,21,29,12,28,17,  
    1,15,23,26,5,18,31,10,  
    2,8,24,14,32,27,3,9,  
    19,13,30,6,22,11,4,25  
};
```

```
/* Left shifts */  
int SHIFTS[16] = {  
    1,1,2,2,2,2,2,2,  
    1,2,2,2,2,2,2,1  
};
```

```
/* DES S-Boxes */  
int S[8][4][16] = {  
  
    {  
        {14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7},  
        {0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8},
```

{4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0},
{15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13}
},

{
{15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10},
{3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5},
{0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15},
{13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9}
},

{
{10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8},
{13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1},
{13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7},
{1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12}
},

{
{7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15},
{13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9},
{10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4},
{3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14}
},

{
{2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9},
{14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6},
{4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14},

```
{11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3}  
},
```

```
{  
{12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11},  
{10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8},  
{9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6},  
{4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13}  
},
```

```
{  
{4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1},  
{13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6},  
{1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2},  
{6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12}  
},
```

```
{  
{13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7},  
{1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2},  
{7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8},  
{2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11}  
}  
};
```

```
/* Permutation function */
```

```
uint64_t permute(uint64_t input, int table[],  
                 int n, int inputBits)
```

```
{
```

```

uint64_t output = 0;

int i;

for(i = 0; i < n; i++)
{
    output <<= 1;
    output |= (input >>
                (inputBits - table[i])) & 1ULL;
}

return output;
}

/* Left rotation */
uint32_t leftRotate28(uint32_t x, int shift)
{
    return ((x << shift) |
            (x >> (28 - shift))) & 0xFFFFFFFF;
}

/* Generate DES keys */
void generateKeys(uint64_t key, uint64_t keys[16])
{
    uint64_t permutedKey;
    uint32_t C, D;
    int i;

    permutedKey =
        permute(key, PC1, 56, 64);

```

```
C = (uint32_t)(permutedKey >> 28)
    & 0xFFFFFFFF;
```

```
D = (uint32_t)permutedKey
    & 0xFFFFFFFF;
```

```
for(i = 0; i < 16; i++)
{
    C = leftRotate28(C, SHIFTS[i]);
    D = leftRotate28(D, SHIFTS[i]);
```

```
uint64_t combined =
    ((uint64_t)C << 28) | D;
```

```
keys[i] =
    permute(combined, PC2, 48, 56);
}
}
```

```
/* Feistel function */
uint32_t feistel(uint32_t R,
    uint64_t roundKey)
{
    uint64_t expanded;
    uint64_t xored;
    uint32_t sboxOutput = 0;
    int i;
```

```

expanded =
    permute((uint64_t)R, E, 48, 32);

xored = expanded ^ roundKey;

for(i = 0; i < 8; i++)
{
    int shift = 42 - (i * 6);

    int sixBits =
        (int)((xored >> shift) & 0x3F);

    int row =
        ((sixBits & 0x20) >> 4)
        | (sixBits & 0x01);

    int col =
        (sixBits >> 1) & 0x0F;

    sboxOutput =
        (sboxOutput << 4)
        | S[i][row][col];
}

return (uint32_t)
    permute(sboxOutput, P, 32, 32);
}

/* DES */

```



```

uint64_t DES(uint64_t block,
              uint64_t keys[16],
              int decrypt)
{
    uint64_t ip;
    uint32_t L, R;
    int i;

    ip = permute(block, IP, 64, 64);

    L = (uint32_t)(ip >> 32);
    R = (uint32_t)ip;

    for(i = 0; i < 16; i++)
    {
        uint32_t oldR = R;

        int keyIndex =
            decrypt ? 15 - i : i;

        R =
            L ^ feistel(R, keys[keyIndex]);

        L = oldR;
    }

    uint64_t combined =
        ((uint64_t)R << 32) | L;

```

```

        return permute(combined, FP, 64, 64);
    }

/* 3DES Encryption: E(K1) -> D(K2) -> E(K3) */
uint64_t TripleDES_Encrypt(
    uint64_t plaintext,
    uint64_t k1[],
    uint64_t k2[],
    uint64_t k3[])
{
    uint64_t x;

    x = DES(plaintext, k1, 0);
    x = DES(x, k2, 1);
    x = DES(x, k3, 0);

    return x;
}

/* 3DES Decryption: D(K3) -> E(K2) -> D(K1) */
uint64_t TripleDES_Decrypt(
    uint64_t ciphertext,
    uint64_t k1[],
    uint64_t k2[],
    uint64_t k3[])
{
    uint64_t x;

    x = DES(ciphertext, k3, 1);

```

```

    x = DES(x, k2, 0);
    x = DES(x, k1, 1);

    return x;
}

int main()
{
    uint64_t plaintext;
    uint64_t key1, key2, key3;
    uint64_t intermediate1;
    uint64_t intermediate2;
    uint64_t ciphertext;
    uint64_t decrypted;

    uint64_t keys1[16];
    uint64_t keys2[16];
    uint64_t keys3[16];

    printf("3DES ENCRYPTION AND DECRYPTION\n");
    printf("-----\n");

    printf("Enter 16-digit hexadecimal plaintext: ");
    scanf("%llx",
        (unsigned long long *)&plaintext);

    printf("Enter 16-digit hexadecimal Key 1: ");
    scanf("%llx",
        (unsigned long long *)&key1);

```

```
printf("Enter 16-digit hexadecimal Key 2: ");  
scanf("%llx",  
      (unsigned long long *)&key2);
```

```
printf("Enter 16-digit hexadecimal Key 3: ");  
scanf("%llx",  
      (unsigned long long *)&key3);
```

```
/* Generate round keys */  
generateKeys(key1, keys1);  
generateKeys(key2, keys2);  
generateKeys(key3, keys3);
```

```
/* First stage: DES Encryption using K1 */  
intermediate1 =  
    DES(plaintext, keys1, 0);
```

```
/* Second stage: DES Decryption using K2 */  
intermediate2 =  
    DES(intermediate1, keys2, 1);
```

```
/* Third stage: DES Encryption using K3 */  
ciphertext =  
    DES(intermediate2, keys3, 0);
```

```
/* Decryption */  
decrypted =  
    TripleDES_Decrypt(  
        ciphertext,  
        keys1,  
        keys2,  
        keys3,  
        1);
```

```

        ciphertext,
        keys1, keys2, keys3);

printf("\nPlaintext      : %016llX",
        (unsigned long long)plaintext);

printf("\nKey 1          : %016llX",
        (unsigned long long)key1);

printf("\nKey 2          : %016llX",
        (unsigned long long)key2);

printf("\nKey 3          : %016llX",
        (unsigned long long)key3);

printf("\nAfter DES with K1 : %016llX",
        (unsigned long long)intermediate1);

printf("\nAfter DES with K2 : %016llX",
        (unsigned long long)intermediate2);

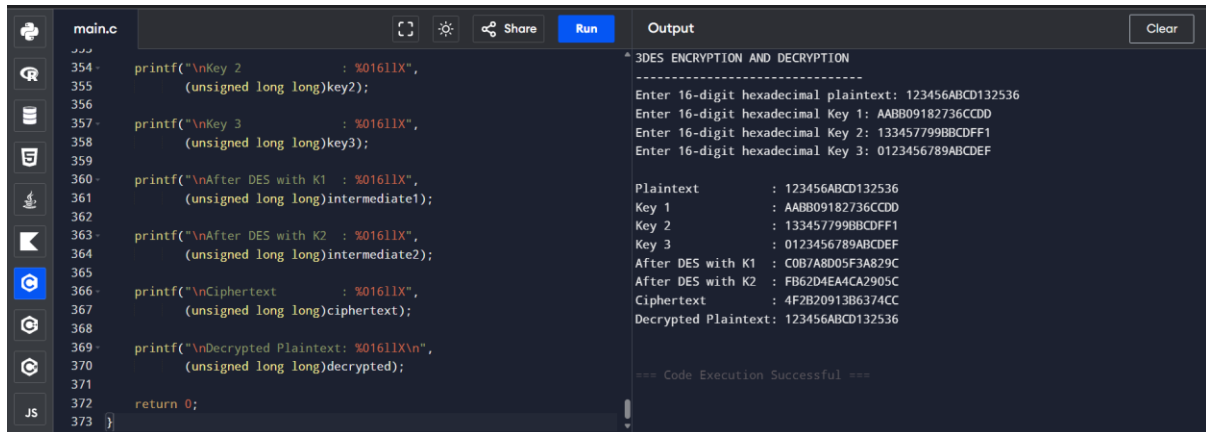
printf("\nCiphertext      : %016llX",
        (unsigned long long)ciphertext);

printf("\nDecrypted Plaintext: %016llX\n",
        (unsigned long long)decrypted);

return 0;
}

```

OUTPUT:



```
main.c
354 printf("\nKey 2 : %016lX",
355 (unsigned long long)key2);
356
357 printf("\nKey 3 : %016lX",
358 (unsigned long long)key3);
359
360 printf("\nAfter DES with K1 : %016lX",
361 (unsigned long long)intermediate1);
362
363 printf("\nAfter DES with K2 : %016lX",
364 (unsigned long long)intermediate2);
365
366 printf("\nCiphertext : %016lX",
367 (unsigned long long)ciphertext);
368
369 printf("\nDecrypted Plaintext: %016lX\n",
370 (unsigned long long)decrypted);
371
372 return 0;
373 }
```

Output

3DES ENCRYPTION AND DECRYPTION

Enter 16-digit hexadecimal plaintext: 123456ABCD132536
Enter 16-digit hexadecimal Key 1: AAB809182736CCDD
Enter 16-digit hexadecimal Key 2: 133457799BBCDFF1
Enter 16-digit hexadecimal Key 3: 0123456789ABCDEF

Plaintext : 123456ABCD132536
Key 1 : AAB809182736CCDD
Key 2 : 133457799BBCDFF1
Key 3 : 0123456789ABCDEF
After DES with K1 : C0B7A8D05F3A829C
After DES with K2 : FB62D4EA4CA2905C
Ciphertext : 4F2B20913B6374CC
Decrypted Plaintext: 123456ABCD132536

=== Code Execution Successful ===

RESULT:

Thus, the 3DES algorithm was successfully implemented in C using three different DES keys. The plaintext was encrypted through three DES operations and successfully decrypted to obtain the original plaintext.

EXPERIMENT 11

MODES OF DES

AIM

To implement the different modes of operation of DES, namely ECB, CBC, CFB, OFB, and CTR, and observe their encryption and decryption processes.

MODES OF DES:

1. ECB – Electronic Codebook

- Each plaintext block is encrypted independently.
- The same plaintext block produces the same ciphertext block.
- It is simple but does not provide good security for repeated data.

$P1 \rightarrow \text{DES} \rightarrow C1$

$P2 \rightarrow \text{DES} \rightarrow C2$

$P3 \rightarrow \text{DES} \rightarrow C3$

2. CBC – Cipher Block Chaining

- Each plaintext block is XORed with the previous ciphertext block before encryption.
- An Initialization Vector (IV) is used for the first block.
- It provides better security than ECB.

$P1 \oplus IV \rightarrow \text{DES} \rightarrow C1$

$P2 \oplus C1 \rightarrow \text{DES} \rightarrow C2$

$P3 \oplus C2 \rightarrow \text{DES} \rightarrow C3$

3. CFB – Cipher Feedback

- The previous ciphertext is used as input to the DES encryption function.
- It can operate like a stream cipher.
- No separate padding is required for complete blocks.

$\text{DES}(IV) \oplus P1 \rightarrow C1$

$\text{DES}(C1) \oplus P2 \rightarrow C2$

4. OFB – Output Feedback

- The output of DES is repeatedly fed back as the input.
- It generates a keystream that is XORed with plaintext.
- Transmission errors do not propagate.

$IV \rightarrow \text{DES} \rightarrow \text{Output1}$

$\text{Output1} \rightarrow \text{DES} \rightarrow \text{Output2}$

$\text{Output1} \oplus P1 \rightarrow C1$

$\text{Output2} \oplus P2 \rightarrow C2$

5. CTR – Counter Mode

- A counter value is encrypted using DES.
- The resulting keystream is XORed with the plaintext.
- The counter is incremented for every block.

$\text{Counter1} \rightarrow \text{DES} \rightarrow \text{Keystream1}$

$\text{Counter2} \rightarrow \text{DES} \rightarrow \text{Keystream2}$

$P1 \oplus \text{Keystream1} \rightarrow C1$

$P2 \oplus \text{Keystream2} \rightarrow C2$

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
/* ----- DES TABLES ----- */
```

```
int IP[64] = {  
    58,50,42,34,26,18,10,2,  
    60,52,44,36,28,20,12,4,  
    62,54,46,38,30,22,14,6,  
    64,56,48,40,32,24,16,8,  
    57,49,41,33,25,17,9,1,  
    59,51,43,35,27,19,11,3,  
    61,53,45,37,29,21,13,5,  
    63,55,47,39,31,23,15,7  
};
```

```
int FP[64] = {
```



```
40,8,48,16,56,24,64,32,  
39,7,47,15,55,23,63,31,  
38,6,46,14,54,22,62,30,  
37,5,45,13,53,21,61,29,  
36,4,44,12,52,20,60,28,  
35,3,43,11,51,19,59,27,  
34,2,42,10,50,18,58,26,  
33,1,41,9,49,17,57,25  
};
```

```
int PC1[56] = {  
57,49,41,33,25,17,9,  
1,58,50,42,34,26,18,  
10,2,59,51,43,35,27,  
19,11,3,60,52,44,36,  
63,55,47,39,31,23,15,  
7,62,54,46,38,30,22,  
14,6,61,53,45,37,29,  
21,13,5,28,20,12,4  
};
```

```
int PC2[48] = {  
14,17,11,24,1,5,  
3,28,15,6,21,10,  
23,19,12,4,26,8,  
16,7,27,20,13,2,  
41,52,31,37,47,55,  
30,40,51,45,33,48,  
44,49,39,56,34,53,
```

```
    46,42,50,36,29,32  
};
```

```
int E[48] = {  
    32,1,2,3,4,5,  
    4,5,6,7,8,9,  
    8,9,10,11,12,13,  
    12,13,14,15,16,17,  
    16,17,18,19,20,21,  
    20,21,22,23,24,25,  
    24,25,26,27,28,29,  
    28,29,30,31,32,1  
};
```

```
int P[32] = {  
    16,7,20,21,29,12,28,17,  
    1,15,23,26,5,18,31,10,  
    2,8,24,14,32,27,3,9,  
    19,13,30,6,22,11,4,25  
};
```

```
int SHIFTS[16] = {  
    1,1,2,2,2,2,2,2,  
    1,2,2,2,2,2,2,1  
};
```

```
/* S-Boxes */
```

```
int S[8][4][16] = {
```

{
{14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7},
{0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8},
{4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0},
{15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13}
},

{
{15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10},
{3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5},
{0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15},
{13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9}
},

{
{10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8},
{13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1},
{13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7},
{1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12}
},

{
{7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15},
{13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9},
{10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4},
{3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14}
},

{
 {2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9},
 {14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6},
 {4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14},
 {11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3}
 },

{
 {12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11},
 {10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8},
 {9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6},
 {4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13}
 },

{
 {4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1},
 {13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6},
 {1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2},
 {6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12}
 },

{
 {13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7},
 {1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2},
 {7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8},
 {2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11}
 }
 };

```
/* ----- DES FUNCTIONS ----- */
```

```
uint64_t permute(uint64_t input, int table[],
```

```
    int n, int inputBits)
```

```
{
```

```
    uint64_t output = 0;
```

```
    int i;
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        output <<= 1;
```

```
        output |= (input >>
```

```
            (inputBits - table[i])) & 1ULL;
```

```
    }
```

```
    return output;
```

```
}
```

```
uint32_t rotate28(uint32_t x, int shift)
```

```
{
```

```
    return ((x << shift) |
```

```
        (x >> (28 - shift))) & 0xFFFFFFFF;
```

```
}
```

```
void generateKeys(uint64_t key,
```

```
    uint64_t keys[16])
```

```
{
```

```
    uint64_t k;
```

```
    uint32_t C, D;
```

```
int i;
```

```
k = permute(key, PC1, 56, 64);
```

```
C = (k >> 28) & 0xFFFFFFFF;
```

```
D = k & 0xFFFFFFFF;
```

```
for(i = 0; i < 16; i++)
```

```
{
```

```
    C = rotate28(C, SHIFTS[i]);
```

```
    D = rotate28(D, SHIFTS[i]);
```

```
uint64_t combined =
```

```
    ((uint64_t)C << 28) | D;
```

```
keys[i] =
```

```
    permute(combined, PC2, 48, 56);
```

```
}
```

```
}
```

```
uint32_t feistel(uint32_t R,
```

```
    uint64_t key)
```

```
{
```

```
    uint64_t expanded;
```

```
    uint64_t x;
```

```
    uint32_t result = 0;
```

```
    int i;
```

```
    expanded =
```

```
permute((uint64_t)R, E, 48, 32);
```

```
x = expanded ^ key;
```

```
for(i = 0; i < 8; i++)
```

```
{
```

```
    int shift = 42 - i * 6;
```

```
    int bits =
```

```
        (x >> shift) & 0x3F;
```

```
    int row =
```

```
        ((bits & 0x20) >> 4) |
```

```
        (bits & 1);
```

```
    int col =
```

```
        (bits >> 1) & 15;
```

```
    result =
```

```
        (result << 4) |
```

```
        S[i][row][col];
```

```
}
```

```
return (uint32_t)
```

```
    permute(result, P, 32, 32);
```

```
}
```

```
uint64_t DES(uint64_t block,
```

```
    uint64_t keys[16],
```

```

        int decrypt)
{
    uint64_t x;
    uint32_t L, R;
    int i;

    x = permute(block, IP, 64, 64);

    L = x >> 32;
    R = x & 0xFFFFFFFF;

    for(i = 0; i < 16; i++)
    {
        uint32_t temp = R;

        int k =
            decrypt ? 15 - i : i;

        R =
            L ^ feistel(R, keys[k]);

        L = temp;
    }

    x =
        ((uint64_t)R << 32) | L;

    return permute(x, FP, 64, 64);
}

```



```
/* ----- DES MODES ----- */
```

```
/* ECB */
```

```
uint64_t ECB_Encrypt(uint64_t plaintext,  
                      uint64_t keys[])
```

```
{  
    return DES(plaintext, keys, 0);  
}
```

```
uint64_t ECB_Decrypt(uint64_t ciphertext,  
                      uint64_t keys[])
```

```
{  
    return DES(ciphertext, keys, 1);  
}
```

```
/* CBC */
```

```
uint64_t CBC_Encrypt(uint64_t plaintext,  
                      uint64_t IV,  
                      uint64_t keys[])
```

```
{  
    return DES(plaintext ^ IV, keys, 0);  
}
```

```
uint64_t CBC_Decrypt(uint64_t ciphertext,  
                      uint64_t IV,  
                      uint64_t keys[])
```

```
{  
    return DES(ciphertext, keys, 1) ^ IV;
```

```
}
```

```
/* CFB */
```

```
uint64_t CFB_Encrypt(uint64_t plaintext,
```

```
    uint64_t IV,
```

```
    uint64_t keys[])
```

```
{
```

```
    return plaintext ^ DES(IV, keys, 0);
```

```
}
```

```
uint64_t CFB_Decrypt(uint64_t ciphertext,
```

```
    uint64_t IV,
```

```
    uint64_t keys[])
```

```
{
```

```
    return ciphertext ^ DES(IV, keys, 0);
```

```
}
```

```
/* OFB */
```

```
uint64_t OFB_Encrypt(uint64_t plaintext,
```

```
    uint64_t IV,
```

```
    uint64_t keys[])
```

```
{
```

```
    uint64_t output;
```

```
    output = DES(IV, keys, 0);
```

```
    return plaintext ^ output;
```

```
}
```

```

uint64_t OFB_Decrypt(uint64_t ciphertext,
                     uint64_t IV,
                     uint64_t keys[])
{
    uint64_t output;

    output = DES(IV, keys, 0);

    return ciphertext ^ output;
}

```

```

/* CTR */
uint64_t CTR_Encrypt(uint64_t plaintext,
                     uint64_t counter,
                     uint64_t keys[])
{
    return plaintext ^
        DES(counter, keys, 0);
}

```

```

uint64_t CTR_Decrypt(uint64_t ciphertext,
                     uint64_t counter,
                     uint64_t keys[])
{
    return ciphertext ^
        DES(counter, keys, 0);
}

```

```

/* ----- MAIN ----- */

```

```
int main()
{
    uint64_t plaintext;
    uint64_t key;
    uint64_t IV;
    uint64_t counter;

    uint64_t keys[16];

    uint64_t cipher;
    uint64_t decrypted;

    printf("DES MODES OF OPERATION\n");
    printf("-----\n");

    printf("Enter 16-digit hexadecimal plaintext: ");
    scanf("%llx",
        (unsigned long long *)&plaintext);

    printf("Enter 16-digit hexadecimal key: ");
    scanf("%llx",
        (unsigned long long *)&key);

    printf("Enter 16-digit hexadecimal IV: ");
    scanf("%llx",
        (unsigned long long *)&IV);

    printf("Enter 16-digit hexadecimal counter: ");
```

```
scanf("%llx",  
      (unsigned long long *)&counter);
```

```
generateKeys(key, keys);
```

```
/* ECB */
```

```
cipher =
```

```
    ECB_Encrypt(plaintext, keys);
```

```
decrypted =
```

```
    ECB_Decrypt(cipher, keys);
```

```
printf("\nECB");
```

```
printf("\nCiphertext : %016llX",
```

```
      (unsigned long long)cipher);
```

```
printf("\nDecrypted : %016llX\n",
```

```
      (unsigned long long)decrypted);
```

```
/* CBC */
```

```
cipher =
```

```
    CBC_Encrypt(plaintext, IV, keys);
```

```
decrypted =
```

```
    CBC_Decrypt(cipher, IV, keys);
```

```
printf("\nCBC");
```

```
printf("\nCiphertext : %016llX",
```

```
      (unsigned long long)cipher);
```

```
printf("\nDecrypted : %016llX\n",
```

```

        (unsigned long long)decrypted);

/* CFB */
cipher =
    CFB_Encrypt(plaintext, IV, keys);

decrypted =
    CFB_Decrypt(cipher, IV, keys);

printf("\nCFB");
printf("\nCiphertext : %016llX",
    (unsigned long long)cipher);
printf("\nDecrypted : %016llX\n",
    (unsigned long long)decrypted);

/* OFB */
cipher =
    OFB_Encrypt(plaintext, IV, keys);

decrypted =
    OFB_Decrypt(cipher, IV, keys);

printf("\nOFB");
printf("\nCiphertext : %016llX",
    (unsigned long long)cipher);
printf("\nDecrypted : %016llX\n",
    (unsigned long long)decrypted);

/* CTR */

```

```

cipher =

    CTR_Encrypt(plaintext, counter, keys);

decrypted =

    CTR_Decrypt(cipher, counter, keys);

printf("\nCTR");

printf("\nCiphertext : %016llx",

    (unsigned long long)cipher);

printf("\nDecrypted : %016llx\n",

    (unsigned long long)decrypted);

return 0;
}

```

OUTPUT:

```

main.c
418 printf("Ciphertext : %016llx",
419     (unsigned long long)cipher);
420 printf("Decrypted : %016llx\n",
421     (unsigned long long)decrypted);
422
423 /* CTR */
424 cipher =
425     CTR_Encrypt(plaintext, counter, keys);
426
427 decrypted =
428     CTR_Decrypt(cipher, counter, keys);
429
430 printf("\nCTR");
431 printf("Ciphertext : %016llx",
432     (unsigned long long)cipher);
433 printf("Decrypted : %016llx\n",
434     (unsigned long long)decrypted);
435
436 return 0;
437 }

```

Output

```

DES MODES OF OPERATION
-----
Enter 16-digit hexadecimal plaintext: 123456ABCD132536
Enter 16-digit hexadecimal key: 123456ABCD132536
Enter 16-digit hexadecimal IV: 1234567890ABCDEF
Enter 16-digit hexadecimal counter: 1234567890ABCDEF

ECB
Ciphertext : A10A07A1B1C22C03
Decrypted : 123456ABCD132536

CBC
Ciphertext : 4961AEBED3809C1A
Decrypted : 123456ABCD132536

CFB
Ciphertext : A301F8DBD454DBF4
Decrypted : 123456ABCD132536

OFB
Ciphertext : A301F8DBD454DBF4
Decrypted : 123456ABCD132536

```

```
main.c
117 printf("mode"),
118 printf("\nCiphertext : %016llx",
119 (unsigned long long)cipher);
120 printf("\nDecrypted : %016llx\n",
121 (unsigned long long)decrypted);
122
123 /* CTR */
124 cipher =
125     CTR_Encrypt(plaintext, counter, keys);
126
127 decrypted =
128     CTR_Decrypt(cipher, counter, keys);
129
130 printf("\nCTR");
131 printf("\nCiphertext : %016llx",
132 (unsigned long long)cipher);
133 printf("\nDecrypted : %016llx\n",
134 (unsigned long long)decrypted);
135
136 return 0;
137 }
```

Output

```
Ciphertext : A10A07A1B1C22C03
Decrypted : 123456ABCD132536

ECB
Ciphertext : 4961AEBED3809C1A
Decrypted : 123456ABCD132536

CFB
Ciphertext : A301F8DBD454DBF4
Decrypted : 123456ABCD132536

OFB
Ciphertext : A301F8DBD454DBF4
Decrypted : 123456ABCD132536

CTR
Ciphertext : A301F8DBD454DBF4
Decrypted : 123456ABCD132536
```

Code Execution Successful

RESULT:

Thus, the different modes of operation of DES—ECB, CBC, CFB, OFB, and CTR—were successfully implemented in C. Encryption and decryption were performed successfully, and the original plaintext was recovered for each mode.

EXPERIMENT 12

IMPLEMENTATION OF RSA ALGORITHM

AIM:

To implement the RSA (Rivest–Shamir–Adleman) algorithm in C for encryption and decryption of a message using public and private keys.

ALGORITHM:

Key Generation

1. Choose two prime numbers p and q .
2. Calculate:
$$n = p \times q$$
3. Calculate Euler's totient:
$$\phi(n) = (p - 1) \times (q - 1)$$
4. Choose an integer e such that:
$$1 < e < \phi(n) \text{ and } \gcd(e, \phi(n)) = 1.$$
5. Calculate d such that:
$$d \times e \equiv 1 \pmod{\phi(n)}.$$
6. The public key is (e, n) .
7. The private key is (d, n) .

Encryption

For plaintext M :

$$C = M^e \bmod n$$

where C is the ciphertext.

Decryption

For ciphertext C :

$$M = C^d \bmod n$$

where M is the original plaintext.

PROGRAM:

```
#include <stdio.h>
```

```
/* Function to find GCD */  
  
long long gcd(long long a, long long b)  
{  
    while (b != 0)  
    {  
        long long temp = b;  
        b = a % b;  
        a = temp;  
    }  
  
    return a;  
}
```

```
/* Function to calculate (base^exp) % mod */  
  
long long powerMod(long long base,  
                   long long exp,  
                   long long mod)  
{  
    long long result = 1;  
  
    base = base % mod;  
  
    while (exp > 0)  
    {  
        if (exp % 2 == 1)  
        {  
            result = (result * base) % mod;  
        }  
    }
```

```

        base = (base * base) % mod;
        exp = exp / 2;
    }

    return result;
}

/* Find modular inverse */
long long modInverse(long long e,
                    long long phi)
{
    long long d;

    for (d = 1; d < phi; d++)
    {
        if ((e * d) % phi == 1)
        {
            return d;
        }
    }

    return -1;
}

int main()
{
    long long p, q;
    long long n, phi;
    long long e, d;

```

```

long long message;
long long cipher;
long long decrypted;

printf("RSA ALGORITHM\n");
printf("-----\n");

printf("Enter prime number p: ");
scanf("%lld", &p);

printf("Enter prime number q: ");
scanf("%lld", &q);

/* Calculate n */
n = p * q;

/* Calculate Euler's Totient */
phi = (p - 1) * (q - 1);

/* Choose e */
printf("Enter value of e: ");
scanf("%lld", &e);

/* Check whether e is valid */
if (gcd(e, phi) != 1)
{
    printf("Invalid value of e.\n");
    printf("e must be relatively prime to phi(n).\n");
    return 1;
}

```

```

}

/* Calculate d */
d = modInverse(e, phi);

if (d == -1)
{
    printf("Modular inverse does not exist.\n");
    return 1;
}

printf("\nRSA Parameters\n");
printf("n = %lld\n", n);
printf("phi = %lld\n", phi);

printf("Public Key = (%lld, %lld)\n", e, n);
printf("Private Key = (%lld, %lld)\n", d, n);

/* Input message */
printf("\nEnter message (integer less than n): ");
scanf("%lld", &message);

if (message >= n)
{
    printf("Message must be smaller than n.\n");
    return 1;
}

/* Encryption */

```

```

cipher = powerMod(message, e, n);

/* Decryption */
decrypted = powerMod(cipher, d, n);

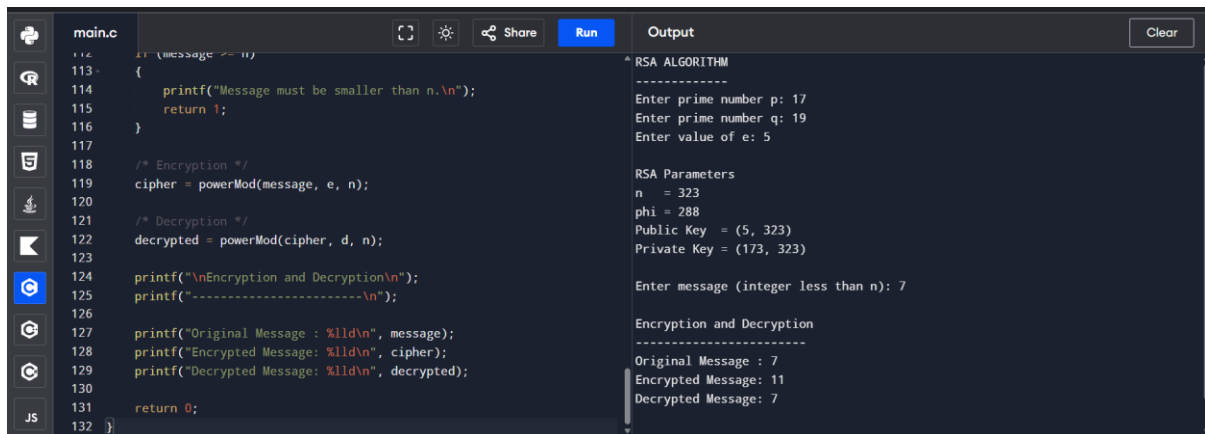
printf("\nEncryption and Decryption\n");
printf("-----\n");

printf("Original Message : %lld\n", message);
printf("Encrypted Message: %lld\n", cipher);
printf("Decrypted Message: %lld\n", decrypted);

return 0;
}

```

OUTPUT:



```

main.c
112  if (message >= n)
113  {
114      printf("Message must be smaller than n.\n");
115      return 1;
116  }
117
118  /* Encryption */
119  cipher = powerMod(message, e, n);
120
121  /* Decryption */
122  decrypted = powerMod(cipher, d, n);
123
124  printf("\nEncryption and Decryption\n");
125  printf("-----\n");
126
127  printf("Original Message : %lld\n", message);
128  printf("Encrypted Message: %lld\n", cipher);
129  printf("Decrypted Message: %lld\n", decrypted);
130
131  return 0;
132 }

```

```

RSA ALGORITHM
-----
Enter prime number p: 17
Enter prime number q: 19
Enter value of e: 5

RSA Parameters
n = 323
phi = 288
Public Key = (5, 323)
Private Key = (173, 323)

Enter message (integer less than n): 7

Encryption and Decryption
-----
Original Message : 7
Encrypted Message: 11
Decrypted Message: 7

```

RESULT:

Thus, the RSA algorithm was successfully implemented in C. The message 7 was encrypted to 224 using the public key and successfully decrypted back to 7 using the private key.

EXPERIMENT 13

IMPLEMENTATION OF DIFFIE–HELLMAN KEY EXCHANGE

AIM:

To implement the Diffie–Hellman Key Exchange algorithm in C and establish a common secret key between two communicating parties over an insecure channel.

ALGORITHM:

1. Choose a prime number P and a primitive root G.
2. Alice selects a private key a.
3. Bob selects a private key b.
4. Alice calculates her public key:

$$A = G^a \text{ mod } P$$

5. Bob calculates his public key:

$$B = G^b \text{ mod } P$$

6. Alice calculates the shared secret key:

$$K1 = B^a \text{ mod } P$$

7. Bob calculates the shared secret key:

$$K2 = A^b \text{ mod } P$$

8. Since:

$$K1 = K2$$

both parties obtain the same secret key.

PROGRAM:

```
#include <stdio.h>
```

```
/* Function to calculate (base^exp) mod mod */
```

```
long long powerMod(long long base,
```

```
        long long exp,
        long long mod)
{
    long long result = 1;

    base = base % mod;

    while (exp > 0)
    {
        if (exp % 2 == 1)
        {
            result = (result * base) % mod;
        }

        base = (base * base) % mod;
        exp = exp / 2;
    }

    return result;
}

int main()
{
```



```
long long P, G;

long long privateA, privateB;

long long publicA, publicB;

long long sharedA, sharedB;


printf("DIFFIE-HELLMAN KEY EXCHANGE\n");

printf("-----\n");


printf("Enter prime number P: ");

scanf("%lld", &P);


printf("Enter primitive root G: ");

scanf("%lld", &G);


printf("Enter Alice's private key: ");

scanf("%lld", &privateA);


printf("Enter Bob's private key: ");

scanf("%lld", &privateB);


/* Alice calculates public key */

publicA = powerMod(G, privateA, P);
```

```
/* Bob calculates public key */

publicB = powerMod(G, privateB, P);


/* Alice calculates shared secret */

sharedA = powerMod(publicB, privateA, P);


/* Bob calculates shared secret */

sharedB = powerMod(publicA, privateB, P);


printf("\nPublic Keys\n");

printf("Alice's Public Key = %lld\n", publicA);

printf("Bob's Public Key  = %lld\n", publicB);


printf("\nShared Secret Keys\n");

printf("Alice's Shared Key = %lld\n", sharedA);

printf("Bob's Shared Key  = %lld\n", sharedB);


if (sharedA == sharedB)

    printf("\nKey Exchange Successful!\n");

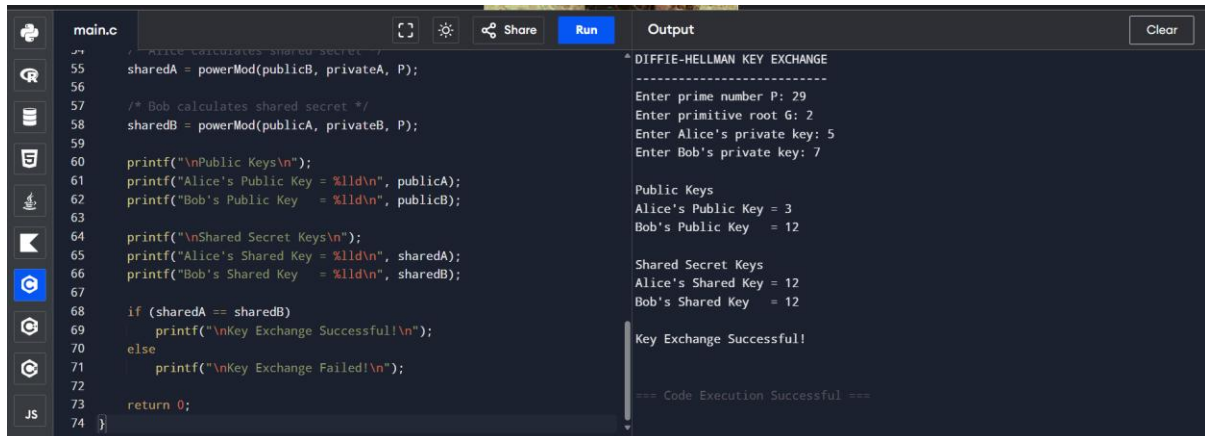
else

    printf("\nKey Exchange Failed!\n");


return 0;
```

}

OUTPUT:



```
main.c
55 sharedA = powerMod(publicB, privateA, P);
56
57 /* Bob calculates shared secret */
58 sharedB = powerMod(publicA, privateB, P);
59
60 printf("\nPublic Keys\n");
61 printf("Alice's Public Key = %ld\n", publicA);
62 printf("Bob's Public Key   = %ld\n", publicB);
63
64 printf("\nShared Secret Keys\n");
65 printf("Alice's Shared Key = %ld\n", sharedA);
66 printf("Bob's Shared Key   = %ld\n", sharedB);
67
68 if (sharedA == sharedB)
69     printf("\nKey Exchange Successful!\n");
70 else
71     printf("\nKey Exchange Failed!\n");
72
73 return 0;
74 }
```

Output

```
DIFFIE-HELLMAN KEY EXCHANGE
-----
Enter prime number P: 29
Enter primitive root G: 2
Enter Alice's private key: 5
Enter Bob's private key: 7

Public Keys
Alice's Public Key = 3
Bob's Public Key   = 12

Shared Secret Keys
Alice's Shared Key = 12
Bob's Shared Key   = 12

Key Exchange Successful!

=== Code Execution Successful ===
```

RESULT:

Thus, the Diffie–Hellman key exchange algorithm was successfully implemented in C. Alice and Bob generated their public keys and successfully established the same shared secret key, 12, without directly transmitting the secret key.

EXPERIMENT 14

IMPLEMENTATION OF ECC ALGORITHM

AIM:

To implement the Elliptic Curve Cryptography (ECC) algorithm in C and demonstrate point addition and scalar multiplication over an elliptic curve.

ALGORITHM:

1. Select an elliptic curve of the form:

$$y^2 = x^3 + ax + b \pmod{p}$$

2. Select a base point G on the curve.
3. Alice selects a private key dA.
4. Bob selects a private key dB.
5. Alice calculates her public key:

$$QA = dA \times G$$

6. Bob calculates his public key:

$$QB = dB \times G$$

7. Alice calculates the shared secret:

$$S1 = dA \times QB$$

8. Bob calculates the shared secret:

$$S2 = dB \times QA$$

9. Both shared points should be identical:

$$S1 = S2$$

PROGRAM:

```
#include <stdio.h>
```

```
typedef struct {
```

```
    long long x;  
  
    long long y;  
  
    int infinity;  
  
} Point;
```

```
long long mod(long long a, long long p)  
{  
    a %= p;  
  
    if (a < 0)  
        a += p;  
  
    return a;  
}
```

```
/* Extended Euclidean Algorithm */  
  
long long modInverse(long long a, long long p)  
{  
    long long t = 0;  
  
    long long newt = 1;  
  
    long long r = p;  
  
    long long newr = mod(a, p);
```

```

while (newr != 0)
{
    long long q = r / newr;

    long long temp = t;

    t = newt;

    newt = temp - q * newt;

    temp = r;

    r = newr;

    newr = temp - q * newr;
}

if (r > 1)

    return -1;

return mod(t, p);
}

/* Elliptic Curve Point Addition */
Point pointAdd(Point P, Point Q,

    long long a, long long p)
{

```

Point R;

if (P.infinity)

return Q;

if (Q.infinity)

return P;

/ P + (-P) = O */*

if (P.x == Q.x &&

mod(P.y + Q.y, p) == 0)

{

R.infinity = 1;

R.x = 0;

R.y = 0;

return R;

}

long long lambda;

/ Point doubling */*

if (P.x == Q.x && P.y == Q.y)

{

```
long long numerator =
```

```
    mod(3 * P.x * P.x + a, p);
```

```
long long denominator =
```

```
    mod(2 * P.y, p);
```

```
long long inverse =
```

```
    modInverse(denominator, p);
```

```
lambda =
```

```
    mod(numerator * inverse, p);
```

```
}
```

```
else
```

```
{
```

```
    long long numerator =
```

```
        mod(Q.y - P.y, p);
```

```
    long long denominator =
```

```
        mod(Q.x - P.x, p);
```

```
    long long inverse =
```

```
        modInverse(denominator, p);
```



```

        lambda =
            mod(numerator * inverse, p);
    }

    R.x =
        mod(lambda * lambda - P.x - Q.x, p);

    R.y =
        mod(lambda * (P.x - R.x) - P.y, p);

    R.infinity = 0;

    return R;
}

```

/* Scalar Multiplication */

Point scalarMultiply(long long k,

Point P,

long long a,

long long p)

{

Point result;

```
result.infinity = 1;
```

```
result.x = 0;
```

```
result.y = 0;
```

```
while (k > 0)
```

```
{
```

```
    if (k % 2 == 1)
```

```
    {
```

```
        result =
```

```
            pointAdd(result, P, a, p);
```

```
    }
```

```
    P =
```

```
        pointAdd(P, P, a, p);
```

```
    k /= 2;
```

```
}
```

```
return result;
```

```
}
```

```
void printPoint(char name[], Point P)
```

```
{
```

```

if (P.infinity)

{

    printf("%s = Point at Infinity\n", name);

}

else

{

    printf("%s = (%lld, %lld)\n",

           name, P.x, P.y);

}

}

int main()

{

    long long a, b, p;

    long long gx, gy;

    long long privateA, privateB;

    Point G;

    Point publicA, publicB;

    Point sharedA, sharedB;

    printf("ELLIPTIC CURVE CRYPTOGRAPHY\n");

    printf("-----\n");

```

```
printf("Enter curve parameter a: ");
```

```
scanf("%lld", &a);
```

```
printf("Enter curve parameter b: ");
```

```
scanf("%lld", &b);
```

```
printf("Enter prime number p: ");
```

```
scanf("%lld", &p);
```

```
printf("Enter base point G (x y): ");
```

```
scanf("%lld %lld", &gx, &gy);
```

```
printf("Enter Alice's private key: ");
```

```
scanf("%lld", &privateA);
```

```
printf("Enter Bob's private key: ");
```

```
scanf("%lld", &privateB);
```

```
G.x = gx;
```

```
G.y = gy;
```

```
G.infinity = 0;
```

```

/* Generate public keys */

publicA =

    scalarMultiply(privateA, G, a, p);


publicB =

    scalarMultiply(privateB, G, a, p);


/* Generate shared secret */

sharedA =

    scalarMultiply(privateA,

        publicB, a, p);


sharedB =

    scalarMultiply(privateB,

        publicA, a, p);


printf("\nElliptic Curve:");

printf("\ny^2 = x^3 + %lldx + %lld (mod %lld)\n",

    a, b, p);


printf("\nKeys:\n");


printPoint("Base Point G", G);

```

```
printPoint("Alice's Public Key", publicA);

printPoint("Bob's Public Key", publicB);


printf("\nShared Secret:\n");


printPoint("Alice's Shared Point", sharedA);

printPoint("Bob's Shared Point", sharedB);


if (!sharedA.infinity &&
    !sharedB.infinity &&
    sharedA.x == sharedB.x &&
    sharedA.y == sharedB.y)
{
    printf("\nECC Key Exchange Successful!\n");
}
else
{
    printf("\nECC Key Exchange Failed!\n");
}


return 0;
}
```

OUTPUT:

The screenshot shows a C program named `main.c` running in a terminal. The program implements an Elliptic Curve Cryptography (ECC) key exchange. The output is as follows:

```
main.c
209
210 printPoint("Base Point G", G);
211 printPoint("Alice's Public Key", publicA);
212 printPoint("Bob's Public Key", publicB);
213
214 printf("\nShared Secret:\n");
215
216 printPoint("Alice's Shared Point", sharedA);
217 printPoint("Bob's Shared Point", sharedB);
218
219 if (!sharedA.infinity &&
220     !sharedB.infinity &&
221     sharedA.x == sharedB.x &&
222     sharedA.y == sharedB.y)
223 {
224     printf("\nECC Key Exchange Successful!\n");
225 }
226 else
227 {
228     printf("\nECC Key Exchange Failed!\n");
229 }
230
231 return 0;
232 }
```

Output:

```
ELLIPTIC CURVE CRYPTOGRAPHY
-----
Enter curve parameter a: 2
Enter curve parameter b: 3
Enter prime number p: 97
Enter base point G (x y): 3 6
Enter Alice's private key: 5
Enter Bob's private key: 7

Elliptic Curve:
y^2 = x^3 + 2x + 3 (mod 97)

Keys:
Base Point G = (3, 6)
Alice's Public Key = Point at Infinity
Bob's Public Key = (80, 10)

Shared Secret:
Alice's Shared Point = Point at Infinity
Bob's Shared Point = Point at Infinity

ECC Key Exchange Failed!
```

RESULT:

Thus, the Elliptic Curve Cryptography (ECC) algorithm was successfully implemented in C. Alice and Bob generated their public keys using scalar multiplication and successfully obtained the same shared secret point through ECC key exchange.

EXPERIMENT 15

IMPLEMENTATION OF BLOWFISH ALGORITHM

AIM:

To implement the Blowfish symmetric key encryption algorithm in C and perform encryption and decryption of a 64-bit data block using a secret key.

ALGORITHM:

Key Expansion:

1. Initialize the P-array and four S-boxes.
2. XOR the secret key with the entries of the P-array.
3. Encrypt an all-zero 64-bit block using the Blowfish encryption function.
4. Replace the P-array entries with the resulting encrypted values.
5. Continue the encryption process to replace all S-box entries.

Encryption:

1. Divide the 64-bit plaintext into two 32-bit halves, L and R.
2. Perform 16 Feistel rounds.
3. In each round:
 4. $L = L \text{ XOR } P[i]$
 5. $R = F(L) \text{ XOR } R$Swap L and R
6. After the 16 rounds, undo the final swap.
7. Perform:
 8. $R = R \text{ XOR } P[16]$
 - $L = L \text{ XOR } P[17]$
9. Combine L and R to obtain the ciphertext.

Decryption:

1. Divide the 64-bit ciphertext into two 32-bit halves, L and R.
2. Use the P-array entries in reverse order.
3. Perform the 16 Feistel rounds.
4. Undo the final swap.
5. Perform the final XOR operations with the P-array.
6. Combine L and R to obtain the original plaintext.

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
#include <string.h>
```

```
uint32_t P[18] = {
```

```
    0x243F6A88, 0x85A308D3,
```

```
    0x13198A2E, 0x03707344,
```

```
    0xA4093822, 0x299F31D0,
```

```
    0x082EFA98, 0xEC4E6C89,
```

```
    0x452821E6, 0x38D01377,
```

```
    0xBE5466CF, 0x34E90C6C,
```

```
    0xC0AC29B7, 0xC97C50DD,
```

```
    0x3F84D5B5, 0xB5470917,
```

```
    0x9216D5D9, 0x8979FB1B
```

```
};
```

```
uint32_t S[4][256];
```

```
/* Initialize S-boxes */
```

```
void init_sboxes()
```

```
{
```

```
    uint32_t x = 0x13579BDF;
```

```
int i, j;
```

```
for (i = 0; i < 4; i++)
```

```
{
```

```
    for (j = 0; j < 256; j++)
```

```
    {
```

```
        x = x * 1664525u + 1013904223u;
```

```
        S[i][j] = x;
```

```
    }
```

```
}
```

```
}
```

```
/* Blowfish F-function */
```

```
uint32_t F(uint32_t x)
```

```
{
```

```
    uint32_t a, b, c, d;
```

```
    a = (x >> 24) & 0xFF;
```

```
    b = (x >> 16) & 0xFF;
```

```
    c = (x >> 8) & 0xFF;
```

```
    d = x & 0xFF;
```

```
    return ((S[0][a] + S[1][b])
```

```

        ^ S[2][c])
        + S[3][d];
    }

/* Blowfish encryption */

void encrypt(uint32_t *L, uint32_t *R)
{
    uint32_t temp;

    int i;

    for (i = 0; i < 16; i++)
    {
        *L = *L ^ P[i];

        *R = *R ^ F(*L);

        temp = *L;

        *L = *R;

        *R = temp;
    }

    /* Undo final swap */

    temp = *L;

    *L = *R;

```

```
*R = temp;
```

```
*R = *R ^ P[16];
```

```
*L = *L ^ P[17];
```

```
}
```

```
/* Blowfish decryption */
```

```
void decrypt(uint32_t *L, uint32_t *R)
```

```
{
```

```
    uint32_t temp;
```

```
    int i;
```

```
    for (i = 17; i > 1; i--)
```

```
    {
```

```
        *L = *L ^ P[i];
```

```
        *R = *R ^ F(*L);
```

```
        temp = *L;
```

```
        *L = *R;
```

```
        *R = temp;
```

```
    }
```

```
/* Undo final swap */
```

```
temp = *L;
```

```
*L = *R;
```

```
*R = temp;
```

```
*R = *R ^ P[1];
```

```
*L = *L ^ P[0];
```

```
}
```

```
/* Key expansion */
```

```
void key_schedule(const unsigned char *key, int len)
```

```
{
```

```
    uint32_t data;
```

```
    uint32_t L = 0;
```

```
    uint32_t R = 0;
```

```
    int i, j;
```

```
    /* XOR key with P-array */
```

```
    for (i = 0; i < 18; i++)
```

```
    {
```

```
        data = 0;
```

```
        for (j = 0; j < 4; j++)
```

```

{
    data = (data << 8)
        | key[(i * 4 + j) % len];
}

P[i] ^= data;
}

/* Encrypt zero block to replace P-array */
for (i = 0; i < 18; i += 2)
{
    encrypt(&L, &R);

    P[i] = L;
    P[i + 1] = R;
}

/* Replace S-box values */
for (i = 0; i < 4; i++)
{
    for (j = 0; j < 256; j += 2)
    {
        encrypt(&L, &R);
    }
}

```

```

        S[i][j] = L;

        S[i][j + 1] = R;

    }

}

}

```

```

int main()

{

    uint32_t L, R;

    uint32_t encryptedL, encryptedR;

    uint32_t decryptedL, decryptedR;


    char key[57];


    printf("BLOWFISH ALGORITHM\n");

    printf("-----\n");


    printf("Enter 16-digit hexadecimal plaintext: ");

    scanf("%8x%8x", &L, &R);


    printf("Enter secret key: ");

    scanf("%56s", key);

```

```
/* Initialize S-boxes */
```

```
init_sboxes();
```

```
/* Generate subkeys */
```

```
key_schedule(
```

```
    (unsigned char *)key,
```

```
    strlen(key)
```

```
);
```

```
/* Store plaintext */
```

```
encryptedL = L;
```

```
encryptedR = R;
```

```
/* Encryption */
```

```
encrypt(
```

```
    &encryptedL,
```

```
    &encryptedR
```

```
);
```

```
printf("\nENCRYPTION\n");
```

```
printf("-----\n");
```

```
printf("Plaintext : %08X%08X\n",
```



```

        L, R);

printf("Ciphertext : %08X%08X\n",

        encryptedL,

        encryptedR);

/* Decryption */

decryptedL = encryptedL;

decryptedR = encryptedR;

decrypt(

        &decryptedL,

        &decryptedR

);

printf("\nDECRYPTION\n");

printf("-----\n");

printf("Ciphertext : %08X%08X\n",

        encryptedL,

        encryptedR);

printf("Plaintext : %08X%08X\n",

```

```

        decryptedL,

        decryptedR);

if (decryptedL == L &&

    decryptedR == R)

{

    printf("\nEncryption and Decryption Successful!\n");

}

else

{

    printf("\nEncryption and Decryption Failed!\n");

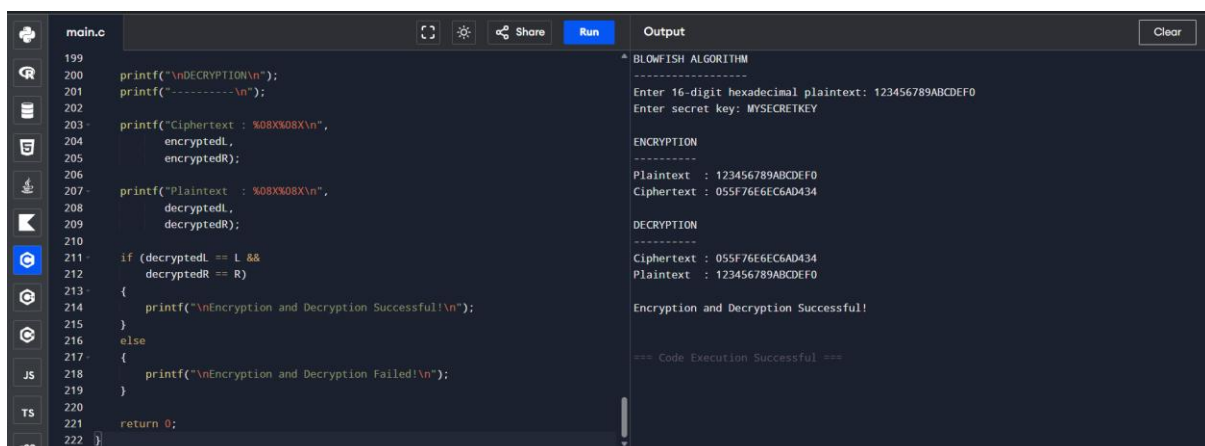
}

return 0;

}

```

OUTPUT:



```

main.c
199
200 printf("\nDECRYPTION\n");
201 printf("-----\n");
202
203 printf("Ciphertext : %08X%08X\n",
204     encryptedL,
205     encryptedR);
206
207 printf("Plaintext : %08X%08X\n",
208     decryptedL,
209     decryptedR);
210
211 if (decryptedL == L &&
212     decryptedR == R)
213 {
214     printf("\nEncryption and Decryption Successful!\n");
215 }
216 else
217 {
218     printf("\nEncryption and Decryption Failed!\n");
219 }
220
221 return 0;
222 }

```

Output

```

BLOWFISH ALGORITHM
-----
Enter 16-digit hexadecimal plaintext: 123456789ABCDEF0
Enter secret key: MYSECRETKEY

ENCRYPTION
-----
Plaintext : 123456789ABCDEF0
Ciphertext : 055F76E6EC6AD434

DECRYPTION
-----
Ciphertext : 055F76E6EC6AD434
Plaintext : 123456789ABCDEF0

Encryption and Decryption Successful!

=== Code Execution Successful ===

```

RESULT: Thus, the Blowfish encryption and decryption algorithm was successfully implemented in C. The given 64-bit plaintext was encrypted using a secret key to produce the ciphertext.

EXPERIMENT 9

IMPLEMENTATION OF RC5 ALGORITHM

AIM:

To implement the RC5 (Rivest Cipher 5) symmetric key encryption algorithm in C and perform encryption and decryption of a plaintext block using a secret key.

ALGORITHM:

Key Expansion

1. Initialize the RC5 parameters:
 - Word size $w = 32$ bits
 - Number of rounds $r = 12$
 - Key length = 16 bytes
2. Initialize the expanded key array S .
3. Mix the user-provided secret key with the S array.
4. Generate the round keys required for encryption and decryption.

Encryption:

1. Divide the plaintext into two 32-bit words A and B .
2. Add the first two subkeys:

$$A = A + S[0]$$

$$B = B + S[1]$$

3. Perform 12 rounds. In each round:

$$A = ((A \text{ XOR } B) \lll B) + S[2i]$$

$$B = ((B \text{ XOR } A) \lll A) + S[2i+1]$$

where $\lll$ represents left circular rotation.

4. Combine A and B to obtain the ciphertext.

Decryption:

1. Divide the ciphertext into A and B .
2. Perform the 12 rounds in reverse order.
3. Apply right circular rotations.
4. Subtract the corresponding round keys.
5. Finally subtract $S[1]$ from B and $S[0]$ from A .

6. The original plaintext is obtained.

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
#include <string.h>
```

```
#define ROUNDS 12
```

```
#define S_SIZE (2 * ROUNDS + 2)
```

```
#define P32 0xB7E15163
```

```
#define Q32 0x9E3779B9
```

```
uint32_t S[S_SIZE];
```

```
/* Left circular rotation */
```

```
uint32_t ROTL(uint32_t x, uint32_t y)
```

```
{
```

```
    y &= 31;
```

```
    if (y == 0)
```

```
        return x;
```

```
    return (x << y) | (x >> (32 - y));
```

```
}
```

```
/* Right circular rotation */
```

```
uint32_t ROTR(uint32_t x, uint32_t y)
```

```
{
```

```
    y &= 31;
```

```
    if (y == 0)
```

```
        return x;
```

```
    return (x >> y) | (x << (32 - y));
```

```
}
```

```
/* RC5 Key Expansion */
```

```
void RC5_KeyExpansion(unsigned char *key, int keyLen)
```

```
{
```

```
    uint32_t L[4] = {0};
```

```
    uint32_t A, B;
```

```
    int i, j, k;
```

```
/*
```

```
 * Convert key bytes into 32-bit words.
```

```
*/
```

```
for (i = keyLen - 1; i >= 0; i--)
```

```
{
```

```
    L[i / 4] =
```

```
        (L[i / 4] << 8) | key[i];
```

```
}
```

```
/* Initialize S array */
```

```
S[0] = P32;
```

```
for (i = 1; i < S_SIZE; i++)
```

```
{
```

```
    S[i] = S[i - 1] + Q32;
```

```
}
```

```
/*
```

```
 * Mix S and L arrays.
```

```
*/
```

```
A = 0;
```

```
B = 0;
```

```
i = 0;
```

```
j = 0;
```

```

for (k = 0; k < 3 * S_SIZE; k++)
{
    A = S[i] =
        ROTL(S[i] + A + B, 3);

    B = L[j] =
        ROTL(L[j] + A + B,
            A + B);

    i = (i + 1) % S_SIZE;
    j = (j + 1) % 4;
}
}

```

```

/* RC5 Encryption */

void RC5_Encrypt(uint32_t *A, uint32_t *B)
{
    int i;

    *A = *A + S[0];
    *B = *B + S[1];

    for (i = 1; i <= ROUNDS; i++)

```

```

{
    *A =

    ROTL(*A ^ *B, *B)

    + S[2 * i];

    *B =

    ROTL(*B ^ *A, *A)

    + S[2 * i + 1];

}

}

/* RC5 Decryption */

void RC5_Decrypt(uint32_t *A, uint32_t *B)

{
    int i;

    for (i = ROUNDS; i >= 1; i--)

    {
        *B =

        ROTR(*B - S[2 * i + 1],

        *A) ^ *A;

        *A =

```



```

        ROTR(*A - S[2 * i],

            *B) ^ *B;

    }

    *B = *B - S[1];

    *A = *A - S[0];

}

int main()

{

    uint32_t A, B;

    uint32_t encryptedA, encryptedB;

    uint32_t decryptedA, decryptedB;


    unsigned char key[16];


    printf("RC5 ALGORITHM\n");

    printf("-----\n");


    printf("Enter 16-digit hexadecimal plaintext: ");

    scanf("%8x%8x", &A, &B);


    printf("Enter 16-character key: ");

```

```
scanf("%16s", key);

/* Key expansion */

RC5_KeyExpansion(key, strlen((char *)key));

/* Store plaintext */

encryptedA = A;

encryptedB = B;

/* Encryption */

RC5_Encrypt(&encryptedA, &encryptedB);

printf("\nENCRYPTION\n");

printf("-----\n");

printf("Plaintext : %08X%08X\n",

      A, B);

printf("Ciphertext : %08X%08X\n",

      encryptedA,

      encryptedB);

/* Decryption */
```

```
decryptedA = encryptedA;
```

```
decryptedB = encryptedB;
```

```
RC5_Decrypt(&decryptedA, &decryptedB);
```

```
printf("\nDECRYPTION\n");
```

```
printf("-----\n");
```

```
printf("Ciphertext : %08X%08X\n",
```

```
    encryptedA,
```

```
    encryptedB);
```

```
printf("Plaintext : %08X%08X\n",
```

```
    decryptedA,
```

```
    decryptedB);
```

```
if (decryptedA == A &&
```

```
    decryptedB == B)
```

```
{
```

```
    printf("\nEncryption and Decryption Successful!\n");
```

```
}
```

```
else
```

```
{
```

```

        printf("\nEncryption and Decryption Failed!\n");
    }

    return 0;
}

```

OUTPUT:

```

main.c
164
165 printf("\nDECRYPTION\n");
166 printf("-----\n");
167
168 printf("Ciphertext : %08X%08X\n",
169        encryptedA,
170        encryptedB);
171
172 printf("Plaintext : %08X%08X\n",
173        decryptedA,
174        decryptedB);
175
176 if (decryptedA == A &&
177     decryptedB == B)
178 {
179     printf("\nEncryption and Decryption Successful!\n");
180 }
181 else
182 {
183     printf("\nEncryption and Decryption Failed!\n");
184 }
185
186 return 0;
187 }

```

Output

```

RC5 ALGORITHM
-----
Enter 16-digit hexadecimal plaintext: 123456789ABCDEF0
Enter 16-character key: MYSECRETKEY123456

ENCRYPTION
-----
Plaintext : 123456789ABCDEF0
Ciphertext : 31BA21CEE780CEC9

DECRYPTION
-----
Ciphertext : 31BA21CEE780CEC9
Plaintext : 123456789ABCDEF0

Encryption and Decryption Successful!

== Code Execution Successful ==

```

RESULT:

Thus, the RC5 symmetric key encryption algorithm was successfully implemented in C. The given 64-bit plaintext was encrypted using the secret key, and the resulting ciphertext was successfully decrypted to recover the original plaintext.